Project Template: Retrieval-Augmented Generation (RAG)

Structured Outline for Presenting a RAG Project

1. Name of Project

RetailPromo Insight Bot

2. Business Details about Project

Business Context:

Retail businesses run dozens of simultaneous promotions spanning multiple types — Fixed Price, Order Threshold, Buy X Get Y, Percentage Discounts, Bundle Deals, and Loyalty Multipliers. These promotions are managed across different product categories, have varied validity windows, and come with complex eligibility rules and restrictions. Store staff, marketing teams, and customer service representatives frequently struggle to quickly locate the correct details for a specific promotion, leading to incorrect pricing at the point of sale, customer dissatisfaction, and missed upselling opportunities.

Goals:

- To provide a centralised, AI-driven interface for querying all active and upcoming retail promotions.
- To enable instant retrieval of promotion details including type, associated products, validity dates, priority, and restrictions.
- To allow store staff and marketing teams to ask natural language questions and receive accurate, document-grounded answers.
- To reduce training time for new staff by replacing manual catalogue lookups with a conversational assistant.

Problem Statement:

Currently, retail staff rely on printed catalogues, spreadsheets, or emails to look up promotion details. There is no centralised system that allows instant, intelligent queries about which promotions are active, what restrictions apply, or which products qualify. This results in incorrect pricing at POS, customer complaints, and lost revenue from promotions not being applied correctly.

Target Stakeholders:

- Store Managers and Frontline Staff applying promotions at POS.
- Marketing and Promotions Teams managing and updating campaigns.
- Customer Service Representatives answering customer queries about active offers.
- Category Managers reviewing promotion performance and planning new campaigns.

Intended Business Impact:

- Reduction in POS errors caused by incorrect or missed promotion application.
- Faster on boarding of new staff through an easy-to-use Q&A interface.
- Improved customer experience through accurate, instant promotion information.
- Increased promotion utilisation rate by making offer details easily accessible.

3. Technical Details about Project

Architecture Overview:

The project leverages a Retrieval-Augmented Generation (RAG) architecture to provide AI-driven Q&A capabilities over retail promotions catalogues. The architecture is designed to efficiently process promotion PDF documents, store embeddings for fast semantic retrieval, and generate accurate, context-grounded answers using the Llama language model via the Groq API.

Components & Data Flow:

Document Ingestion:

- **Input:** PDF documents containing retail promotion details — promotion type, identifier, title, enabled status, associated products, priority, start/end dates, and restrictions.
- **Processing:** Documents are read using pypdf. Text is extracted page by page, cleaned to remove noise, excess whitespace, and artefacts from PDF parsing.

Chunking:

- Large promotion catalogues are split into smaller, semantically coherent chunks of approximately 500 characters with 100-character overlap between adjacent chunks.
- The overlap ensures that promotion details that span a chunk boundary are not lost during retrieval.

Vectorization & Indexing:

- Each text chunk is converted into a 128-dimensional vector embedding using the Llama model via Groq API.
- Embeddings capture the semantic meaning of each promotion entry, enabling similarity-based search rather than exact keyword matching.
- FAISS (Facebook AI Similarity Search) is used as the vector database to store all embeddings, providing fast nearest-neighbour retrieval at query time.

Retrieval Module:

- When a user query is submitted, it is first converted into an embedding using the same Llama model.
- FAISS retrieves the top 5 most semantically similar chunks based on L2 distance scores.

- Retrieved chunks serve as the grounded context for the generation step, ensuring answers are based solely on catalogue data.

Generation Module:

- The top retrieved chunks plus the user's question are sent to Llama (llama-3.3-70b-versatile via Groq) as a structured prompt.
- The model is instructed to answer only from the provided context and to clearly state when information is not available.
- Temperature is set to 0.2 to ensure consistent, factual, low-creativity responses.

User Interaction / Q&A Interface:

- Users interact via a terminal-based chat loop.

I. Ask specific questions: e.g. "What are the restrictions on the Spend $80 get $10 off promotion?" or "List all Buy X Get Y promotions active this month."

II. Request summaries: e.g. "Summarise all Order Threshold promotions currently enabled."

- The system dynamically retrieves relevant promotion chunks and generates human-readable responses.

Main Algorithms & Models:

- **Embedding Generation:** Llama model (via Groq) produces 128-dimensional vector representations of text chunks.
- **Vector Similarity Search:** FAISS IndexFlatL2 provides fast k-nearest neighbour search over all stored promotion embeddings.
- **RAG Model:** Combines FAISS retrieval results with Llama generation to produce accurate, document-grounded answers.
- **Prompt Engineering:** A carefully crafted system prompt restricts the model to answer only from provided context, preventing hallucination.

Technical Considerations:

Scalability:

III. FAISS supports large-scale indexing and can handle thousands of promotion chunks across multiple catalogues.

IV. The index is persisted to disk (faiss_index.bin) so it does not need to be rebuilt on every run.

Security & Access Control:

V. Promotion catalogues may contain commercially sensitive pricing and campaign strategies.

VI. Access to the application and the stored index files should be restricted to authorised staff only.

Data Privacy:

VII. No customer data is processed by this system.
VIII. Groq API calls transmit text chunks to a third-party cloud service — ensure this aligns with organisational data policies.

Performance Optimisation:

IX. Embeddings are generated once during indexing and cached in FAISS — queries do not require re-embedding all documents.
X. The 'rebuild' command allows selective re-indexing when catalogues are updated, without restarting the application.

4. Implementation (in 5 Steps)

- **Step 1 - Initial Setup / Data Ingestion:** Extract text from retail promotion PDF catalogues using pypdf, clean the text by removing whitespace and artefacts, and split into overlapping chunks of ~500 characters for processing.

- **Step 2 - Retrieval Process:** Generate 128-dimensional vector embeddings for each text chunk using the Llama model via the Groq API, then store all embeddings in a FAISS IndexFlatL2 index persisted to disk for fast similarity-based retrieval.

- **Step 3 - Generation Step:** Convert the incoming user query into an embedding, retrieve the top 5 most relevant promotion chunks from FAISS, and pass them as grounded context to Llama via Groq to generate a concise, accurate answer.

- **Step 4 - Integration / User Interface:** Provide a terminal-based chat loop where retail staff can submit natural language queries or request promotion summaries, with 'rebuild' and 'quit' command support for catalogue updates and graceful exit.

- **Step 5 - Deployment / Monitoring:** Deploy the system on a local workstation or internal server, set the GROQ_API_KEY environment variable, monitor query accuracy, update PDF catalogues when new promotions are launched, and trigger a rebuild to refresh the FAISS index.

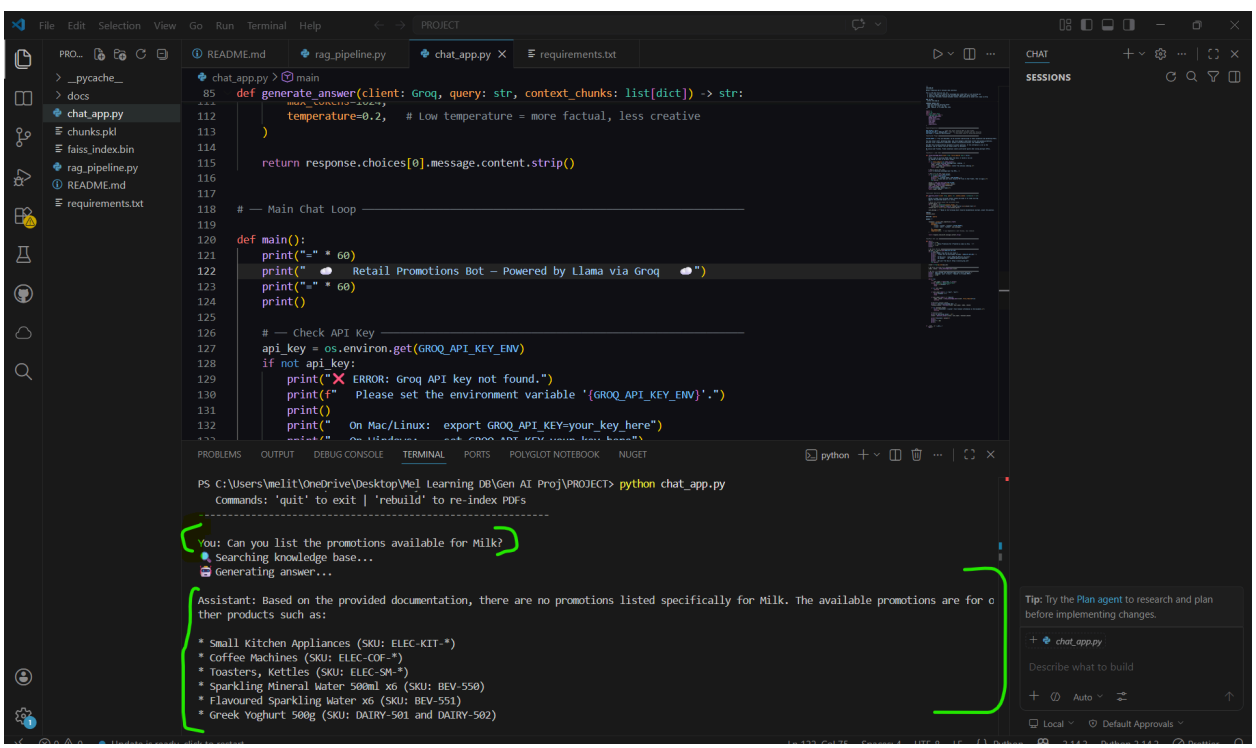
5. Sample Input and Output

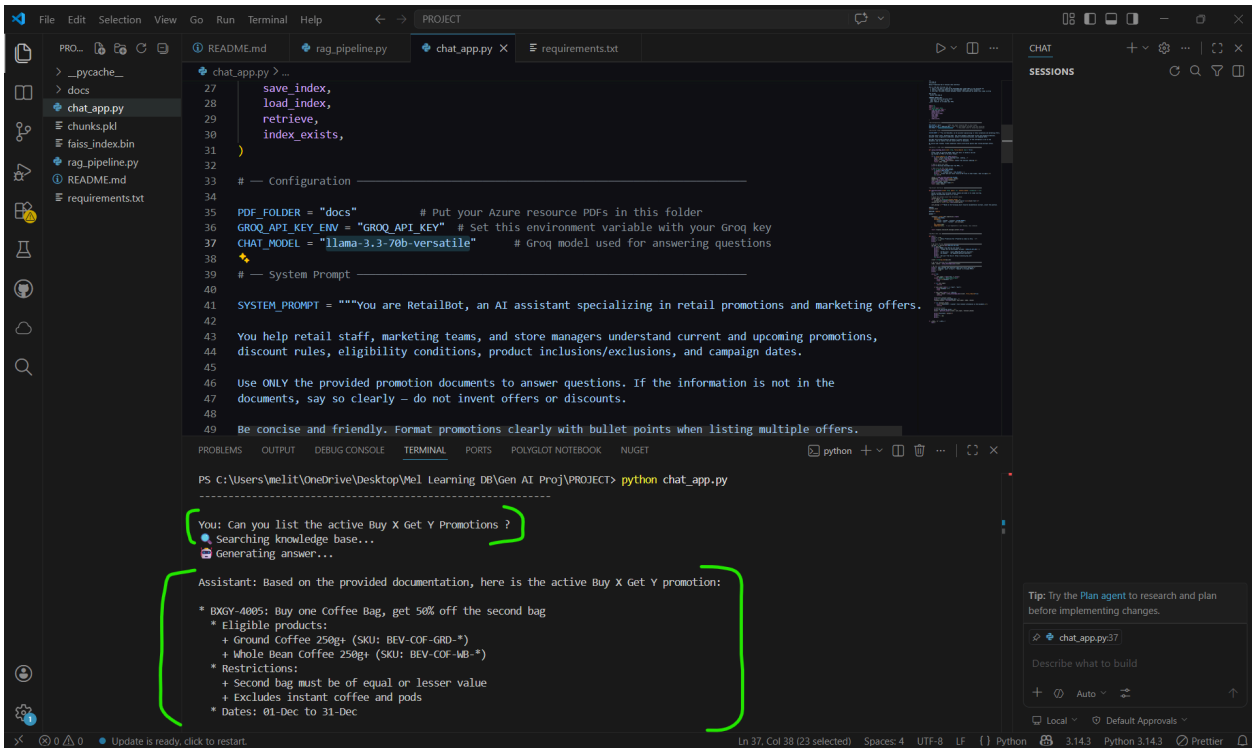
Sample Input:

A PDF file (retail_promotions.pdf) containing the full promotions catalogue for FreshMart Supermarkets, Winter 2024–25 season. The document includes 35 promotions across 6 types: Fixed Price, Order Threshold, Percentage Discount, Buy X Get Y, Bundle Deal, and Loyalty Points Multiplier. Each promotion entry contains the following fields:

- Promotion Type (e.g. Fixed Price, Order Threshold)
- Identifier (e.g. FP-1001, OT-2003)
- Title (e.g. "Spend $80 — Get $10 Off")
- Enabled (Yes / No)
- Associated Products (product names and SKU codes)
- Priority (1 = highest)
- Start Date and End Date
- Restrictions (eligibility rules, limits, exclusions)

Sample Output:





6. Tech Stack

Component	Technology / Detail
Programming Language	Python 3.10+
Language Model	Llama 3 (via Groq API — llama-3.3-70b-versatile)
Vector Database / Retrieval	FAISS (Facebook AI Similarity Search)
Data Processing / Chunking	Python libraries — pypdf, re (regex)
LLM API Client	Groq Python SDK
Storage	Local file system for PDFs, .bin and .pkl for index
User Interface	Terminal / Command-line chat

7. One Best Benefit for Business

Elimination of Errors and Improved Promotion Accuracy:

By providing retail staff with a centralised, AI-driven interface for querying promotion details in natural language, the RetailPromo Insight Bot eliminates the reliance on printed catalogues and manual lookups that are prone to human error. A frontline staff member can simply ask "Is the Buy 2 Get 1 cereal deal still active?" or "What are the restrictions on the bundle discount?" and receive an instant, accurate, document-grounded answer — without leaving the POS terminal or searching through multi-page PDFs.

This directly reduces incorrect pricing at checkout, minimises customer disputes, and ensures that every eligible promotion is applied correctly every time. For a retail business running 30+ simultaneous promotions across hundreds of SKUs, this improvement in operational accuracy translates directly into higher customer trust, reduced refund processing, and measurable revenue protection.